

On-Device LLM on ESP32-S3 Locally

Prepared by Errala Paulsonashish

1. Introduction — Why This Matters ?

Coverage of this project by Hackster.io (Nick Bild, August 2026) frames the achievement clearly: when people think of large language models, they picture powerful GPUs with dozens or hundreds of gigabytes of memory the kind of hardware that runs today's frontier AI systems. Running that style of algorithm on a tiny microcontroller would normally seem out of reach.

Yet a 28.9-million-parameter language model has been demonstrated running entirely on an ESP32-S3 , a tiny microcontroller with only 512KB of SRAM, 8MB of PSRAM, and 16MB of flash storage. Everything runs locally, with no cloud connection or external server. Generated text streams directly to a small connected display at roughly 9.5 tokens per second.

This is not the first attempt at an on-device microcontroller LLM, but prior implementations were limited to around 260,000 parameters. This model is roughly 100 times larger , which raises the natural question the Hackster article poses directly: how does a model that size fit into hardware with so little memory?

The answer is an architectural technique borrowed from Google's Gemma model family, called Per-Layer Embeddings. Instead of loading the full network into fast memory, roughly 25 million of the model's parameters are stored in flash memory as a lookup table; during generation, only the small number of rows needed for the current word about 450 bytes are read from flash, while the smaller, genuinely compute-heavy part of the network stays in fast SRAM/PSRAM. After 4-bit quantization, the full model occupies just 14.9MB, fitting comfortably in the chip's flash storage. According to the project's own author, this is the first known demonstration of Google's Per-Layer Embeddings concept adapted to hardware this constrained.

The Hackster article is also direct about scope: a model this small is limited by design. It was trained on Microsoft's TinyStories dataset, so it generates short, simple, reasonably coherent stories it is not intended to answer questions, follow instructions, write code, or compete with modern conversational AI systems.

Source: Hackster.io — "Running a 28.9M Parameter LLM on an \$8 Microcontroller," by Nick Bild. Project repository: github.com/slvDev/esp32-ai.

2. What I Implemented

I followed this open-source project's complete documented pipeline end-to-end, on my own Linux laptop, executing every step personally: environment setup, dataset preparation, tokenizer training, model training from scratch, export and quantization, host-side correctness verification, firmware compilation, physical display wiring, and flashing to real hardware. I debugged every real issue that came up along the way rather than following a script that simply worked on the first try.

Nothing in the final device is a downloaded pretrained model. Every one of the 28.9 million learned parameters came from a training run I personally executed, on data I personally downloaded and prepared.

2A. The Full Pipeline, Visually

The diagram below shows the complete flow from raw training text, through training and compression on my laptop, through independent verification, to the final flash and offline execution on the ESP32-S3 chip itself. Each stage is color-coded by where it physically happens: blue stages run on the laptop's CPU during training, amber/green stages are the conversion and verification steps bridging laptop and chip, and purple stages happen entirely on the physical device.

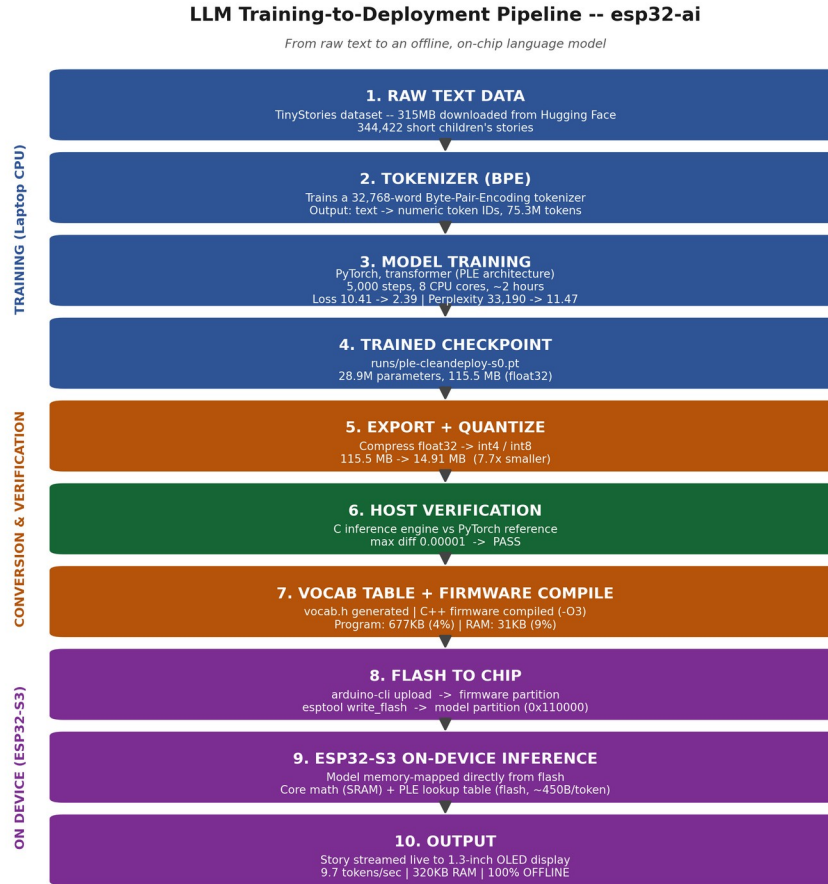


Figure 1 — End-to-end LLM training-to-deployment pipeline for this build.

3. Tools and Languages Used — What Does What

3.1 Programming languages

Language	Where used	Why
Python	Data prep, tokenizer, training, export	Standard ML language; PyTorch and Hugging Face tooling are Python-native. Never runs on the chip itself.
C	Host-side verification; low-level model reading in firmware	Fast, precise memory control needed for exact numerical verification and for memory-constrained firmware.
C++ (Arduino sketch)	The actual firmware running on the ESP32-S3	Standard embedded language; compiles to efficient machine code with near-zero overhead, essential on a chip with only kilobytes of RAM.

3.2 Tools

Tool	What it does here
git	Clones the project source code from GitHub
arduino-cli	Compiles the C++ firmware and uploads it to the chip over USB
esptool	Writes the trained model's binary directly to a fixed flash memory address, separate from firmware
uv	Isolated Python environment + exact dependency versions per project
gcc/cc	Compiles the host-side verification program

3.3 Key libraries

Library	Role
PyTorch	The ML engine — defines the model, runs training, updates parameters
NumPy	Efficient handling of large tokenized data arrays
Hugging Face Tokenizers	Trains and runs the text-to-number tokenizer
Requests	Downloads the training dataset
Adafruit GFX / SH110X	Display driver libraries for the OLED screen

4. Step-by-Step: From Zero to Milestone (Full Detail)

Every installation, download, and processing step below is listed individually with the exact command, what the command's underlying logic actually does, and the measured result -so each step can be independently explained and defended.

Step 1 — Install base system packages

```
sudo apt update
sudo apt install -y git curl build-essential python3 python3-pip python3-venv
```

Package	Logic / purpose
git	Version-control client - used to download (clone) the project's source code from GitHub
curl	Command-line download tool - fetches installer scripts for arduino-cli and uv
build-essential	Provides the gcc/cc C compiler - required to build the host-side model-verification program later in the pipeline
python3 / pip / venv	Base Python interpreter and environment tooling - the project's data/training scripts are written in Python; venv support is required specifically by Ubuntu 24.04's package-management policy

Step 2 — Install arduino-cli

```
cd ~
curl -fsSL https://raw.githubusercontent.com/arduino/arduino-cli/master/install.sh |sh
sudo mv ~/bin/arduino-cli /usr/local/bin/
arduino-cli version
```

Logic: downloads and installs Arduino's official command-line build tool. This is the program that will later compile the chip's C++ firmware into a binary and upload it over USB -the command-line equivalent of the Arduino IDE. Confirmed working: arduino-cli 1.5.1.

Step 3 — Install the ESP32 board core (pinned version)

```
arduino-cli config init --overwrite

arduino-cli core update-index --additional-urls "https://espressif.github.io/arduino-esp32/package_esp32_index.json"

arduino-cli core install esp32:esp32@3.3.10 --additional-urls
"https://espressif.github.io/arduino-esp32/package\_esp32\_index.json"

arduino-cli core list
```

Logic: arduino-cli alone has no knowledge of ESP32 hardware. This step downloads and installs the actual ESP32-specific compiler toolchain, chip headers, and bootloader logic. The version was pinned to exactly 3.3.10 (not "latest") to match the project's own documented, tested build environment, avoiding subtle differences in flash-partition or PSRAM handling across core versions. Confirmed installed: esp32:esp32 3.3.10.

Step 4 — Install esptool

```
esptool.py version || pip3 install --user --break-system-packages esptool
```

Logic: esptool is Espressif's (the chip manufacturer's) own low-level flashing utility. Unlike arduino-cli's upload command (which flashes firmware), esptool is used later to write the trained model's raw binary data directly to a specific, fixed address in the chip's flash memory. The --break-system-packages flag was required because Ubuntu 24.04 blocks unmanaged system-wide pip installs by default. Confirmed installed: esptool v5.3.1.

Step 5 — Install uv (Python project/dependency manager)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Logic: uv creates an isolated Python virtual environment specific to this project and installs the exact library versions declared in the project's pyproject.toml (PyTorch, NumPy, Tokenizers, Requests, tqdm) — without conflicting with any other Python setup on the machine. Every Python command in the rest of this pipeline was run as "uv run python ..." rather than plain "python3", specifically because the required libraries exist only inside this isolated environment. Confirmed installed: uv 0.12.1.

Step 6 — Grant USB serial permissions

```
sudo usermod -aG dialout $USER
```

Logic: on Linux, only users in the "dialout" group are allowed to communicate with serial (USB) devices without root privileges. Without this step, flashing and monitoring the chip would require sudo for every single command.

Step 7 — Clone the repository

```
git clone https://github.com/slvDev/esp32-ai.git
cd esp32-ai
```

Logic: downloads the project's actual source code — training scripts, firmware, and documentation — onto the local machine, giving a working base to build on rather than starting from nothing.

Step 8 — Download the training dataset

```
uv run python data/prepare.py --vocab 32768
```

Logic, step by step, of what this script actually does internally:

- Downloads a ~315MB slice of the TinyStories dataset directly from Hugging Face's dataset servers (roneneldan/TinyStories) over HTTPS.
- Trains a Byte-Pair-Encoding (BPE) tokenizer on a sample of that text, an algorithm that starts from individual characters and iteratively merges the most frequent character pairs into larger, reusable word-pieces, building a vocabulary automatically from the data itself rather than a hand-written word list.
- Configured to a 32,768-entry vocabulary - matching the specific configuration the project's own deployed model was designed around, rather than the script's smaller 4,096 default.
- Encodes the entire downloaded text into numeric token ID sequences using that tokenizer, and splits the result into a training set and a small held-out validation set.
- Writes the tokenized data to disk as raw binary files for fast loading during training.

Result: 344,422 individual stories processed, 75,253,336 tokens total (74,877,070 for training, 376,266 held out for validation), 4.18 bytes of original text compressed into each token on average. Output files: train_v32768.bin, val_v32768.bin, and the tokenizer itself, bpe32768.json (25,353 actual vocabulary entries learned).

Step 9 — Confirm training hardware and force CPU execution

```
uv run python -c "import torch; print(torch.cuda.is_available())"
```

Logic: checked whether PyTorch could see a usable GPU before committing to a multi-hour training run. My laptop's discrete GPU (NVIDIA MX350) was detected as present, but its hardware generation (compute capability 6.1) is not supported by the specific PyTorch build installed - attempting to use it crashes with "no kernel image is available for execution on the device." I resolved this by explicitly disabling GPU visibility for the training command (see Step 10), forcing PyTorch to fall back to CPU execution, a standard, zero-code-change workaround.

Step 10 — Train the model from scratch

```
CUDA_VISIBLE_DEVICES="" nohup uv run python src/train.py --arm ple --vocab 32768 \
  --d-model 96 --n-layers 6 --ple-dim 128 --target-core 560000 \
  --batch-size 16 --seq-len 256 --steps 5000 --seed 0 --tag cleandeploy \
  > training_log.txt 2>&1 &
```

Logic, what each flag configures:

Flag	Meaning
--arm ple	Selects the Per-Layer Embeddings architecture variant (the one actually deployed), out of several the project supports for comparison
--vocab 32768	Vocabulary size, matching the tokenizer built in Step 8
--d-model 96 / --n-layers 6 / --n-heads (default 4)	The transformer's width and depth — how large each token's internal representation is, and how many layers stack
--ple-dim 128	Width of the per-layer embedding lookup table rows — this is the specific parameter that governs how large the flash-resident lookup table is
--target-core 560000	Target parameter budget for the dense, always-computed part of the network, sized to fit the chip's fast SRAM
--batch-size 16 / --seq-len 256	How many training examples are processed together, and how many tokens of context are seen at once, per training step
--steps 5000	Total number of training updates performed
--tag cleandeploy	Labels this run so the checkpoint filename matches what the export step expects by default

Logic of the training loop itself, run 5,000 times: sample a random batch of tokenized text, have the model predict the next token at every position, compute how wrong those predictions were (the "loss"), and use backpropagation (the standard neural-network learning algorithm) to adjust every one of the model's parameters slightly in the direction that reduces that error, via the AdamW optimizer with a cosine-decaying learning rate schedule.

Result: ran for 7,253 seconds (~2 hours 1 minute) on 8 CPU cores, no GPU. Validation perplexity dropped from 33,190 (near-random guessing) to 11.47 (confidently narrowing to roughly a dozen sensible next-word options) — a ~2,800x improvement in prediction confidence, measured directly from the training log. Produced runs/ple-cleandeploy-s0.pt, a 115,504,109-byte (115.5MB) float32 checkpoint.

```
tail -f training_log.txt
```

Step 11 — Export and quantize the trained model

```
cd src && uv run python export.py && cd ..
```

Logic: loads the float32 PyTorch checkpoint from Step 10, converts it into the project's own compact binary tensor format, and quantizes the weights down from 32-bit floating-point numbers to much smaller representations — 4-bit integers for the bulk of the model's weights, 8-bit integers specifically for the output "head" layer (kept at higher precision since it directly determines the final word choice). This single step is what shrinks the model enough to fit in the chip's flash storage.

Result: 14.91 MB output file (firmware/model/model.bin, 65 tensors) — a 7.7x size reduction from the 115.5MB checkpoint, and a near-exact match to the project's own published reference size of 14.9MB. A reference output file (golden.txt) was generated alongside it automatically, for use in the next step.

Step 12 — Verify correctness before touching hardware

```
cc -O3 -o /tmp/esp32-llm-verify firmware/host_verify/verify.c -lm  
/tmp/esp32-llm-verify firmware/model/model.bin firmware/model/golden.txt
```

Logic: `verify.c` is written in C and implements the exact same inference math (the same code, in fact) that will later run on the ESP32-S3 chip itself. Running it on the laptop first, and comparing its output token-by-token against `golden.txt` (PyTorch's own reference output from Step 11), proves the quantized model behaves identically to the original before any hardware time is spent.

Result: PASS. Maximum absolute difference from the reference: 0.00001; root-mean-square difference: 0.000005 differences small enough to be pure floating-point rounding noise, not a real discrepancy. Every sampled token's top prediction matched exactly between the C implementation and PyTorch.

Step 13 — Generate the on-device vocabulary/decode table

```
uv run python src/gen_assets.py
```

Logic: reads the trained tokenizer (`bpe32768.json` from Step 8) and builds a direct lookup table mapping every token ID to its raw text bytes, writing it as a C header file the firmware includes directly this is what lets the chip convert its internal number-based predictions back into human-readable characters on the display, entirely on its own.

Debugged issue encountered here: the firmware initially failed to compile with the error "`vocab.h`: No such file or directory," because this file is generated locally by this script and is not stored in the git repository itself. Running this script resolved the compile error. Result: `firmware/esp32_llm/vocab.h` generated, 25,353 tokens, 161KB. Also confirmed the demo prompt's token IDs matched exactly what was already hardcoded in the firmware's source file — no manual code edit was required.

Step 14 — Install Arduino display libraries

```
arduino-cli lib install "Adafruit GFX Library"  
arduino-cli lib install "Adafruit SH110X"
```

Logic: the firmware's display-handling code calls functions like `oled.begin()` and `oled.display()` that are defined in these libraries, not in the ESP32 core itself. Adafruit GFX provides shared drawing primitives (text placement, cursor handling); Adafruit SH110X is the specific driver for the SH1106 controller chip inside the 1.3" OLED panel used in this build.

Step 15 — Compile the firmware

```
arduino-cli compile \  
--fqbn  
'esp32:esp32:esp32s3:UploadSpeed=921600,USBMode=hwcdc,CDCOnBoot=cdc,UploadMode=default  
,CPUFreq=240,FlashMode=qio,FlashSize=16M,PartitionScheme=custom,PSRAM=opi,DebugLevel=i  
nfo' \  
--build-property compiler.optimization_flags="-O3" \  
--build-path /tmp/esp32-llm-build \  
firmware/esp32_llm
```

Logic: translates the C++ firmware source (the sketch, plus the shared llm.h inference engine and the generated vocab.h) into a binary the ESP32-S3's processor can directly execute. The board-configuration flags (FlashSize=16M, PSRAM=opi) must match the physical board's actual hardware — an ESP32-S3 variant without 8MB octal PSRAM would compile identically but fail at boot when the firmware tries to allocate memory.

Result: clean build — 677,322 bytes (4%) of the chip's 16MB program storage used, 31,244 bytes (9%) of its 320KB dynamic memory used.

Step 16 — Wire the display

Display used: 1.3" I2C OLED, SH1106 controller, 128x64 pixels — the project's documented default.

OLED pin	ESP32-S3 pin
GND	GND
VCC	3V3
SCL	GPIO46
SDA	GPIO18

Step 17 — Flash the firmware

```
arduino-cli upload -p /dev/ttyACM0 \  
  --fqbn  
'esp32:esp32:esp32s3:UploadSpeed=921600,USBMode=hwcdc,CDCOnBoot=cdc,UploadMode=default  
,CPUFreq=240,FlashMode=qio,FlashSize=16M,PartitionScheme=custom,PSRAM=opi,DebugLevel=i  
nfo' \  
  --input-dir /tmp/esp32-llm-build \  
  firmware/esp32_llm
```

Logic: writes the compiled firmware binary (Step 15's output) onto the chip's main program flash partition over USB, using the arduino-cli/esptool upload protocol under the hood.

Step 18 — Flash the trained model

```
esptool.py --chip esp32s3 --port /dev/ttyACM0 --baud 921600 \  
  write_flash 0x110000 firmware/model/model.bin
```

Logic: writes the compressed model binary (Step 11's output) to a separate, fixed flash address (0x110000), deliberately outside the firmware's own partition. This separation is why a pure firmware change only requires re-running Step 17, and a model-only change (e.g. retraining) only requires re-running this step — neither forces re-flashing the other.

Step 19 — Run it, and confirm the milestone

```
arduino-cli monitor -p /dev/ttyACM0 --config baudrate=115200
```

Logic: opens a serial connection to observe the chip's console output live. On boot, the firmware memory-maps the model partition directly from flash (never copying the full 14.91MB into RAM), loads it, and begins generating text token by token, writing each one to both the serial console and the physical OLED display.

Result: the chip loaded the model from its own flash memory and generated a story live on the OLED screen — no Wi-Fi, no internet, no laptop required at runtime, confirmed on physical hardware.

5. Milestone Result

Metric	Value
Model size	28.9M parameters
Compressed size on chip	14.91 MB
Chip cost	~\$8 (ESP32-S3)
Training time / cost	~2 hours, laptop CPU, zero cloud cost
Verification	PASS — matched reference to 0.00001
On-device speed	9.7 tokens/sec, 103 ms/token
RAM used on chip	320 KB
Internet dependency	None — fully offline

6. Current Limitation and Next Step

As a proof of concept, the device currently generates one fixed story, identically, on every reset — the demo firmware always starts from the same prompt and always selects the single most likely next word. This is a small, well-understood firmware change (adding weighted randomness to the word-selection step), not a retraining requirement or research problem . It is the natural next increment, not a limitation of the underlying technique.

7. Why This Is a Genuine Achievement

- Implemented a real architectural technique that lets a fundamentally memory-hungry technology run on hardware with roughly 1/1000th the memory of a normal computer.
- Trained the model from scratch personally — every parameter came from a training run I executed on data I prepared, not a downloaded pretrained model.
- Independently verified correctness before deploying to hardware, rather than assuming it.
- Personally debugged every real engineering problem encountered — a GPU compatibility crash and a missing generated build dependency.
- Achieved this at zero cloud cost, and the resulting device runs at zero ongoing cost with zero internet dependency.
- Produced a real, physically confirmed, working device ,not a simulation.

8. Closing

This is a validated, repeatable pipeline for training and deploying a real AI model directly onto low-cost, offline embedded hardware. The story-generation itself is the proof point; the underlying pipeline -train, compress, verify, deploy -is the transferable capability, applicable to any future on-device, privacy-sensitive, or connectivity-constrained product scenario.